

Evaluating Agent Skills for Version-Specific Plugin Migration: A Retrospective Study

Author information supplied separately

Review draft: 17 September 2026

Abstract

Agent skills package version-specific maintenance knowledge into reusable instructions and tools. Their evaluation must distinguish rewarded diagnosis from satisfaction of the target migration contract. We present a retrospective study of a plugin-upgrade skill on 16 static tasks, with two attempts per condition and an archive of 64 answers and 328 criterion decisions. Mean recorded reward rises from 93.83 to 98.75, a gain of 4.92 points with a task-bootstrap interval of [0.31, 10.86]. Eight task pairs are at the ceiling; excluding the largest-gain task reduces the gain to 2.42 points. Contract-stratified accounting locates changes in original credit. A separate non-blind AI-assisted review of ten answers identifies three criterion disagreements in two answers; replacement intervals touch or cross zero. Exhaustive sign enumeration yields a two-sided tail fraction of 0.0781 for the original endpoint under conditional sign symmetry. Mechanism checks sharpen the case findings: the archived containment predicate accepts the parent directory under both POSIX and Windows path algorithms, while a reconstructed timer experiment shows that explicit teardown and timer unreferencing permit exit under different lifecycle conditions. The latter distinguishes a functioning alternative repair from compliance with a narrower rubric. The study contributes a traceable maintenance evaluation connecting aggregate reward, answer-level contracts, reproducible mechanism checks, and grading sensitivity. Historical configurations provide descriptive context. The checks do not execute complete agent-produced migrations; operational repair success and cross-project generalization remain untested.

Keywords: software maintenance; plugin migration; agent skills; retrospective evaluation; LLM-as-a-judge

1 Introduction

Software migration requires more than replacing names in source code. A repair must respect the contract of the target version, distinguish public APIs from internal implementation, and preserve runtime behavior. Plugin systems make these requirements particularly visible: a change can affect loading, authentication, rendering, event dispatch, or process shutdown independently. Advice that sounds consistent with a release note may still recommend an unavailable API or omit a boundary condition.

An agent skill bundles instructions, reference material, and sometimes executable tools for repeated use. Such a package can reduce the effort needed to locate version-specific facts. It can also introduce stale instructions, unnecessary work, or confidence in an incomplete repair. Existing skill benchmarks already demonstrate heterogeneous benefits and overhead (Li et al., 2026; Han et al., 2026; Yang and Ding, 2026). Consequently, the contribution of a maintenance case study cannot rest only on showing that a skill sometimes helps.

The practical question is whether an observed score gain identifies advice a maintainer can act on. An answer can name the correct migration concept while leaving the required operation unavailable, an edge case unhandled, or a runtime condition unmet. Aggregate reward does not reveal which of these distinctions was checked.

We investigate the shipped plugin-upgrade skill for DSH, linking task contracts to archived answers and the judgments that reward them. The available evidence accumulated during development rather than under one prospectively controlled experiment. We retain this history, distinguish incompatible protocols, and give greatest analytical attention to a later comparison with archived outputs, per-criterion judgments, and execution records. The study asks:

1. RQ1: What changes in recorded reward accompany skill availability in the focal comparison, and how are they distributed across tasks?
2. RQ2: Where does recorded contract credit change, and what do inspected answers reveal about the adequacy of migration advice and its grading?
3. RQ3: How do resource accounting and grading sensitivity qualify the apparent benefit?

The study makes three contributions. First, it provides an auditable within-configuration comparison linking 64 answers to version-pinned tasks, original criterion judgments, and resource records. Second, it combines complete contract-stratified accounting with a separate bounded review, showing both where recorded credit changes and where selected judgments fail to capture an operational requirement. Third, it adds reproducible predicate and timer checks to distinguish an observed defect from an alternative repair excluded by a narrower rubric, and derives concrete review questions from these and the static API case. These contributions concern evaluation and maintenance practice in one setting; the cases do not establish failure prevalence or a new general law of skill effectiveness.

2 Related Work

Skills and software engineering. SkillsBench evaluates skills across diverse tasks (Li et al., 2026). SWE-Skills-Bench examines their utility in software engineering (Han et al., 2026). SkillLens studies skill generation and consumption (Huang et al., 2026), while WebDev-Skills-Bench compares skill conditions in web development, including controls for supplied context (Yang and Ding, 2026). These studies make heterogeneous effectiveness and resource overhead established questions. Our narrower contribution is an artifact-traceable account of version-contract errors and evaluation sensitivity in one maintained plugin-migration setting. We do not replicate their breadth or their controlled ablations.

Version-sensitive maintenance. VersiCode studies version-controllable code generation, and CODE-MENV studies code migration (Wu et al., 2024; Cheng et al., 2025). General software-agent evaluations such as SWE-bench and AgentBench emphasize task and environment design (Jimenez et al., 2024; Liu et al., 2024). Our setting combines migration diagnosis with historical executable tasks, but the most completely archived comparison consists of static answers. Its reward therefore cannot be treated as an executable repair success rate.

Measurement validity. LLM judges enable scalable evaluation but require validation (Zheng et al., 2023); self-preference is a documented concern (Panickssery et al., 2024). This matters when solvers and judges share a model family. Development exposure is a separate issue: repeated use of the same task during skill refinement does not create a fresh holdout (Golchin and Surdeanu, 2024). Our audit addresses selected observable errors and provenance. It cannot eliminate grader bias, task-selection bias, or exposure effects.

3 Setting and Evidence

3.1 Versioned migration and the intervention

DSH is a plugin-based framework with a marketplace and frequent releases. Relevant changes include renamed exports, authentication boundaries, build-artifact layouts, session representations, and version-routing rules. Tasks specify an initial state, a target contract, permitted operations, and a deliverable. Static tasks request diagnosis; hands-on tasks require a repair in a version-pinned environment. A host cold boot checks liveness, while task-specific probes exercise additional requirements. Neither establishes correctness beyond the checked behaviors.

Upgrade cards record symptoms, causes, actions, version applicability, and provenance. For example, card `DSH-0.1.2-A1-01` concerns removal of the Host APIProxy surface on the 0.1.1-rc.2 to 0.1.2-alpha.1 edge. Correct migration requires locating actual call sites and selecting the corresponding Remote interfaces, rather than applying a client-plane recipe to every plugin. This illustrates why merely mentioning the correct release or card is insufficient.

The shipped skill links inspection, version-corridor analysis, implementation, and verification. Cards can inform both the skill and the benchmark. Thus the comparison is an open-book evaluation of the complete package, including access to facts, rather than an isolated test of procedural reasoning. Historical repository states differ, so the intervention is not assumed byte-identical across configurations.

3.2 Task construction and exposure

Contributors turn migration incidents and version changes into cards, fixtures, instructions, reference solutions, and graders. Contributor self-check and maintainer review constitute development quality control, not independent blinded annotation. The repository’s exposure ledger distinguishes feedback used to refine the skill, grader development, and uncleared exposure. We make no claim that the evaluated tasks form an independent holdout.

An immutable earlier snapshot contains 23 tasks (10 static and 13 hands-on). This inventory is not the denominator of every experiment. Historical configurations use 56, 23, 21, or 22 scored tasks. The focal comparison uses a recorded selection of 16 static tasks. Repeated attempts are nested within tasks; they do not increase the number of independent migration problems. Related tasks may also share incident sources, limiting task-level uncertainty estimates.

3.3 Evidence tiers

We distinguish three evidence tiers. The focal comparison answers the research questions; the other tiers document the available context.

1. Focal archived comparison: 16 tasks, two conditions, and two repeats, with 64 reports, original verdicts, scores, selection and execution records, and an offline integrity audit.
2. Historical context: five within-configuration paired summaries, with heterogeneous task pools, aggregation, grading, and raw-artifact coverage.
3. Supplementary evidence: a newer 16-task Qwen comparison with scored records but without archived original answers and judgment reasons, plus three GLM-5.3 rounds with a change of judge in round three. Neither is pooled into the focal estimate.

A complete set of files improves auditability; it does not make a retrospective selection prospective or establish the validity of its grader.

4 Method

4.1 Focal comparison and estimand

The focal run is the archived GLM-5.3-Flash unified S16 comparison. The model label is the recorded gateway label, not an independently verified served-weight identity. Each task has two no-skill and two with-skill outputs, run through the ZCode subagent host. The conditions share a prompt template, but workspace paths differ. This is not a container-isolated or byte-identical-prompt experiment. The selection script draws 16 of S1–S22 using seed 20260915, stratifying by problem family and fixture-size tercile with proportional quotas. Balance constraints require at least two hard tasks, two Chinese-prompt tasks, and two large-fixture tasks. The selector reads metadata and fixture sizes rather than scores; it excludes S3, S4, S7, S8, S10, and S16. The selected set contains five static-contract diagnosis, seven runtime/client API, three release/install, and one profile/Cordis task; 14 prompts are English and two are Chinese. This reproducible draw is within a development-exposed pool, not a random sample of migration problems in general. The schedule alternates arm order in task blocks and reverses it in round two; the recorded execution order is retained. Four pilot cells on excluded S4 and S10 are not included in the 64 formal cells. Formal execution used no hard wall-time cap, and all 64 cells produced scored reports.

Let y_{tar} be the original score for task t , arm a , and repeat r . We calculate $\bar{y}_{ta} = (y_{ta1} + y_{ta2})/2$, $d_t = \bar{y}_{t,skill} - \bar{y}_{t,no}$, and $\hat{\Delta} = 16^{-1} \sum_t d_t$. Thus tasks receive equal weight. The focal percentile bootstrap resamples paired tasks 10,000 times with mulberry32 seed 20260915. It quantifies variability over the observed tasks under resampling assumptions, not uncertainty over arbitrary repositories or model versions.

The report-judge-v2 rubric assigns a weight to each criterion. A pass receives its full weight, partial receives half, and fail or missing receives zero; declared triggered caps can limit the total. Criteria and frozen reference excerpts are supplied to the judge, which returns decisions and reasons rather than a total. The deterministic scorer calculates the 0–100 reward. Proposed verification can satisfy a static task; describing a test does not establish that it was executed.

The same model family supplies semantic evaluation. The run records describe arm-blind staging with one report per judgment; our later review was not blind. The archive retains original verdicts, including historical transport metadata; corrected provenance records distinguish harness-declared judge identity from independently observed provider identity. Scores are recomputed from rubric decisions. An integrity audit checks all 64 report hashes and all 328 original criterion decisions. Two stored reports require reversal of documented relative-link rewriting to reproduce their original hashes; this transformation is explicit rather than silently ignored.

4.2 Bounded output review

The initial targeted review was AI-assisted and non-blind, separate from the original verdicts. An earlier purposive pass covered five complete answers selected for prominent effects or operational concerns. A subsequent selection was committed before reading the additional full reports: paired answers for a positive task (S6), a negative task (S18), and a zero-change task (S12), at fixed repeats. One answer overlapped with the earlier pass. The combined scope is ten unique answers and 56 criteria.

The initial reviewer had access to task rubrics, reports, and earlier judgments, and selection used known outcomes. The authors subsequently reported human checking with broadly similar assessments; the later repository revision identifies contributing plugin authors as non-blind reviewers. The ten archived answers retain the same 56 decisions and replacement scores across these revisions. No separate per-reviewer scoring sheet or item-level confirmation was supplied, so revision stability is not interpreted as measured human–AI agreement or independent blinded validation. Both the initial AI-assisted analysis and the reported human follow-up are retained in the provenance ledger. Selection, report hashes, criterion-level reasons,

and replacement scores are recorded. We keep original results as the primary recorded endpoint and show both the earlier and the combined replacement sensitivities. Replacements affect only reviewed answers; unreviewed scores are not presumed correct.

4.3 Complete contract-stratified analysis

We map each of the focal tasks’ existing rubric criteria to one primary domain: version/release applicability; API/data/ownership; lifecycle/ordering/deployment; safety/boundary handling; failure attribution; or evidence/verification/provenance. The mapping is retrospective and fixed before generating the domain table. Multi-part criteria receive one primary label to avoid double counting, so these categories are an analytical organization of the rubric rather than an independently validated failure taxonomy.

Every formal report is linked to its task packet and original criterion verdicts through stored hashes. The analysis accounts for all 64 reports and all 328 criterion decisions, including original successes and shortfalls. We report full-credit counts with per-arm criterion denominators and distinct task counts. These are recorded judge outcomes, not newly verified correctness labels. This full-cohort accounting is distinct from the targeted ten-answer review; it does not represent an independent semantic regrading of all 64 answers. Repeats are nested within tasks, and tasks can contribute to multiple domains. We make no domain-level significance claims.

4.4 Bounded mechanism and robustness checks

After case selection and the original analysis, we added offline checks under a recorded retrospective protocol. For S11, we extracted the archived proposed containment function, removing only TypeScript annotations, and exercised seven declared lexical inputs under each of Node’s POSIX and Windows path algorithms. Parent and ancestor escapes, ordinary and dotted descendants, prefix siblings, and platform-specific controls distinguish the observed defect from ordinary valid behavior. This does not execute an HTTP route, native Windows filesystem operations, or symlink resolution.

For S18, the fixture describes a timer planner but does not contain its implementation. We therefore reconstructed a minimal self-rearming timer in separate Node child processes. Five conditions compare referenced timers, cleanup that is not invoked, explicit unmount with cancellation, unreferencing every timeout, and unreferencing only the first timeout while another handle initially keeps the process alive. Each child emits a readiness marker; a parent watchdog terminates children still alive 700 milliseconds later. Natural exit is distinguished from watchdog termination. These are mechanism checks with controls, not executions of the original plugin or new scored solver attempts.

For the original and both existing replacement endpoints, we also enumerate all $2^8 = 256$ sign assignments of the eight nonzero paired task differences. The two-sided tail fraction counts assignments whose absolute sum is at least the observed absolute sum, including ties. This is a descriptive sensitivity under independent, sign-exchangeable task differences; the deterministic arm schedule does not justify a causal randomization test. We additionally report every leave-one-task-out mean and both repeat-wise means. None of these endpoints is selected by its significance.

5 Results

5.1 RQ1: Recorded reward changes

In the focal comparison, mean original reward increases from 93.83 to 98.75 (Table 1). Six tasks improve, two decline, and eight have zero change; all eight zero-change pairs are at 100 in both arms. S1 has a 42.5-point task-level gain. Removing it reduces the overall gain to 2.42 points across the remaining 15 tasks.

Focal endpoint	No skill	Skill	Δ	95% task-bootstrap interval
Original judgments	93.83	98.75	4.92	[0.31, 10.86]
Earlier S11 replacement	93.83	98.44	4.61	[−0.23, 10.63]
All reviewed replacements	93.05	98.44	5.39	[0.00, 11.80]

Table 1: Focal 16-task comparison. Original scores remain unchanged in the archive. Replacement rows are partial-review sensitivity analyses, not validated estimates from a fully regraded dataset.

Primary domain	Tasks	Criteria/arm	No skill	Skill
Version and release applicability	6	12	8	12
API, data and ownership contracts	13	46	35	45
Lifecycle, ordering and deployment	9	26	24	25
Safety and boundary handling	5	10	10	10
Failure attribution	11	32	32	32
Evidence, verification and provenance	14	38	31	36

Table 2: Complete contract-stratified analysis of the focal 64 reports. The final two columns count original full-credit criterion decisions; they are not independently validated correctness counts. Criteria/arm includes both repeats. Multi-contract criteria receive one retrospective primary label; tasks can contribute to multiple domains.

The original signed-rank approximation gives $p = 0.0797$ with eight nonzero pairs. The corresponding exhaustive sign-enumeration tail fraction is $20/256 = 0.0781$. Repeat-wise mean gains are 4.53 and 5.31; all 16 leave-one-task-out means range from 2.42 to 5.58. These checks show a positive descriptive direction across these reductions, without strong evidence against a symmetric zero-centered effect. The positive bootstrap lower endpoint should not be portrayed as uniformly strong inferential support.

The focal estimate describes this configuration and task set. Historical and supplementary configurations are reported separately in Appendix B; their different task pools, hosts, and judges preclude treating them as replications of this effect.

5.2 RQ2: Contract failures and grading disagreements

Table 2 includes the full cohort. Original full-credit counts rise from 8 to 12 out of 12 version-related decisions and from 35 to 45 out of 46 API/data/ownership decisions per arm. Failure attribution has full credit in both arms (32/32), as does the safety/boundary domain (10/10). These counts describe the original judge; the reviewed S11 containment defect demonstrates why an all-full-credit domain cannot be called verified safe. Domains contain different tasks and weights, so their percentages are not a difficulty ranking or an estimate of a causal skill mechanism.

The bounded targeted review finds three criterion disagreements in two of ten answers. Because the sample is purposive and non-blind, these counts are not an error-rate estimate. They identify concrete weaknesses in the recorded measurement.

Information access and public API boundaries. S1’s reviewed answers retain their original 55 and 100 scores. Part of its rubric rewards mapping to supplied migration cards. The large gain therefore includes access to organized facts and identifiers, and cannot isolate a procedural reasoning benefit. In S6, the no-skill answer correctly recommends removing obsolete defensive code but incorrectly attributes informational-event handling wholly to the host. It also recognizes a public append API gap while recommending an unqualified append route. Review changes two criteria and lowers this answer from 87.5 to 62.5. The

paired skill answer retains 100 by distinguishing producer responsibilities, retention, and unavailable public surfaces. The resulting sensitivity increases estimated skill benefit, illustrating that grading errors can favor either arm. Additional full-text inspection of S6 repeat 2 finds the same platform-only interpretation in the no-skill answer, whereas both skill answers distinguish retention from a supported producer seam. This is repetition within one task, not a second independent incident; no additional score replacement is imposed in the fixed ten-answer sensitivity.

Containment boundaries. In S11, a with-skill answer proposes a relative-path check equivalent to accepting an empty relative path or any non-absolute path that does not start with two dots followed by a separator. The exact relative path `..` passes this predicate, even though it denotes the parent directory. The extracted predicate accepts the parent in both path algorithms, contradicting the declared containment expectation in two of 14 diagnostic inputs; the other 12 control inputs behave as expected. These deliberately selected inputs are not a security failure-rate estimate. Review lowers the containment criterion from full to partial credit and the answer from 90 to 80. This establishes a defect in the proposed guard; it does not establish a complete exploitable endpoint or claim that every deployment has the same exposure.

Cleanup versus process liveness. For S18 repeat 2, the no-skill answer retains 100 and the skill answer retains 90. The skill answer proposes explicit host teardown, while the rubric requires timers that do not keep a still-mounted probe host alive. Our child-process reconstruction makes the distinction observable: a referenced chain and an uninvoked cleanup callback remain alive until the watchdog stops them; both explicit unmount with cancellation and unreferencing every timeout exit naturally. Unreferencing only the first timeout is insufficient when an initially live companion handle allows a referenced successor to be scheduled.

Thus explicit teardown is a functioning alternative in the reconstruction when the host performs it. The retained partial credit measures noncompliance with the rubric’s narrower mounted-host requirement, not proof that the proposed cleanup can never repair a CI hang. The instruction asks for a one-line hang fix, whereas the rubric specifies timer unreferencing. This difference exposes sensitivity to how acceptable repairs are defined. We preserve the original and prior reviewed scores; the new mechanism check does not silently create another replacement endpoint.

Ceiling and a retained partial score. Both reviewed S12 answers retain 100: they connect a Windows file lock to the native host process, distinguish the recorded release channels, and prescribe the pinned installation and restart sequence. This zero-change pair illustrates a ceiling case rather than evidence that the skill was consumed or unnecessary in all contexts. The previously reviewed S17 no-skill answer retains 80 because recognizing onboarding requirements does not repair an incorrect injection descriptor. No frequency or causal mechanism is inferred from these selected examples.

5.3 RQ3: Sensitivity and resource use

Replacing only the reviewed S11 score changes the focal estimate to 4.61 points and moves its interval across zero. Replacing all reviewed scores changes it to 5.39 points with a lower endpoint at zero. The direction remains positive, but the strength of evidence depends on grading decisions. The exhaustive sign-enumeration tail fractions are $34/256 = 0.1328$ for the S11-only replacement and $30/256 = 0.1172$ for all reviewed replacements, compared with 0.0781 originally. Neither the larger replacement estimate nor the original positive interval should be selected in isolation.

The 64 formal execution records contain 3,185,993 no-skill and 12,320,379 skill `subagent_tokens`, a ratio of 3.87. The corresponding sums of recorded task durations are 9,446 and 11,450 seconds, a ratio of

1.21. The token field does not separately define input, output, and cache accounting, so it is not a billing-cost measure. Summed task durations are not end-to-end wall-clock time under overlapping execution. Pilot records are excluded.

Together, these results answer RQ3 with a conditional assessment: higher recorded reward accompanies greater recorded token use in the focal configuration, while the interval's relation to zero depends on selected grading decisions. The archive supports neither a monetary cost-benefit estimate nor a claim that the added resource use produces more successful repairs.

6 Discussion

6.1 What contract inspection adds to aggregate reward

The focal result has two complementary interpretations. Skill availability is associated with a higher mean recorded score, with much of the gain concentrated in one task and no room for improvement on eight others. Inspection then asks whether the rewarded advice meets the particular requirement that makes a migration usable. This second question matters even when the average improves: S11 receives full containment credit for a predicate that accepts the parent directory, whereas S18's partial credit depends on requiring exit without host teardown. The mechanism check shows that teardown can also permit exit when performed. The cases distinguish a missed predicate defect from a narrower choice of acceptable lifecycle repair.

Full-cohort accounting and targeted inspection answer different questions. The former shows where the original judge assigned credit across every focal answer, avoiding a results table built solely from selected successes or failures. The latter supplies counterexamples with reasons, but cannot estimate how often such defects occur. For example, the safety/boundary domain has full original credit in both arms even though the S11 check fails a concrete boundary case. Neither the domain count nor the selected defect alone characterizes the reliability of the whole dataset.

6.2 Review questions grounded in the observed cases

The cases motivate three checks for maintainers reviewing migration advice. For each proposed API action, verify that the target version exposes a callable public surface and identify the component responsible for producing or retaining the data (S6). For a boundary predicate, test the exact boundary value as well as ordinary descendants; the parent path itself supplies a counterexample in S11. For lifecycle advice, state the required runtime condition separately from its suggested cleanup action; releasing a component resource and allowing process exit are different requirements (S18).

These checks are specific enough to apply to an answer without assuming that its total score establishes correctness. Their proposed use in other projects is a transfer of review questions, not an estimate that the same defects occur there. The S11 predicate was executed directly, and S18 was examined through a minimal timer reconstruction with positive and negative controls. S6 remains a static assessment against the recorded contract; no mock API is substituted for the actual target implementation. We have not tested whether adding these checks to an agent or grader improves subsequent outcomes.

6.3 Implications for evaluating maintenance skills

A maintenance-skill evaluation should state which endpoint it measures: locating applicable knowledge, supplying contract-consistent advice, or completing a verified repair. The focal archive measures graded static advice. S1's card-mapping reward also captures access to organized facts; it cannot isolate procedural reasoning. Reporting task-level changes and the contract behind each selected shortfall makes these distinctions visible without treating the rubric as a substitute for execution.

In this archive, retaining original verdicts alongside explicit alternative scores is necessary to interpret the result. Correcting the reviewed S6 judgment increases estimated skill benefit, while correcting S11 reduces it. Thus review should inspect both arms and preserve disagreements that strengthen or weaken the apparent effect. Our replacement analyses demonstrate this sensitivity locally. S18 additionally shows why a grader should state whether alternative repairs are acceptable or whether a particular lifecycle condition is mandatory. Independent grading and end-to-end repair checks would address uncertainties that these predicate and timer probes leave unresolved.

The historical ordering does not establish a capability-response curve. Baseline reward shares mathematical components with the gain and is bounded by the same ceiling; model comparisons also mix infrastructure, graders, task exposure, and skill versions. The present contribution is an evidence-linked maintenance case study. Claims about an inverted U, production reliability, or cost effectiveness would require additional designs and endpoints.

7 Threats to Validity

Construct validity. Static diagnostic reward is not functional repair success. Some criteria reward card mappings or metadata. Keyword and semantic graders measure different things, and numeric normalization does not harmonize their constructs. The focal solver and judge share a model family. The initial non-blind AI-assisted review is useful for finding counterexamples but does not certify all scores. Human follow-up is author-reported without a separate item-level scoring dataset, precluding an inter-rater agreement estimate. Full-cohort contract tabulation inherits the original judge decisions and should not be confused with complete revalidation. The S18 instruction and rubric differ in how narrowly they specify the hang repair; compliance with the latter is not equivalent to the absence of other functioning repairs.

Internal validity. The data are retrospective, and contributors developed the skill and tasks in an overlapping workflow. Historical selections are not assumed random, and the focal seeded draw does not remove development exposure. Historical execution budgets, environment failures, and protocols differ. Original model labels rely on recorded infrastructure declarations. The review selection was frozen before additional full-output inspection, but after outcomes were known; it is not preregistration of the underlying experiment.

Statistical conclusion validity. There are only 16 focal tasks and two repeats per arm. Eight task pairs are at the ceiling. Task-level resampling treats tasks as independent even when incident sources may overlap. The selected review does not estimate population error, and partial replacements leave unreviewed scoring uncertainty unresolved. Intervals, signed-rank approximations, and leave-one-out results are exploratory descriptions of the recorded data rather than confirmatory tests of a universal effect.

External validity and reproducibility. The focal tasks concern one plugin framework and static reports. Historical task inventories and artifacts have unequal coverage. The newer Qwen outputs and grading reasons are unavailable in the repository, limiting replication of its endpoint. Live models and dependencies can change even with pinned task sources. The archive supports deterministic recalculation of the focal scores and reported sensitivities, not a guarantee of identical future model outputs. Mechanism probes are restricted to an extracted lexical predicate and a reconstructed timer; they do not execute complete agent-produced migrations, and the bounded timer observation is not a production liveness guarantee.

8 Conclusion

In the archived 16-task comparison, skill availability is associated with a 4.92-point increase in recorded reward. Ceiling effects, one large task gain, and grading sensitivity limit the strength and scope of that result. The more actionable finding is that migration advice must be inspected at the contract it is meant to satisfy: API availability and responsibility, exact boundary coverage, and process liveness cannot be inferred from a plausible recommendation or an aggregate score. Complete accounting of 328 original decisions establishes the coverage of the analysis; the separate ten-answer review provides concrete disagreements and retained judgments, not a population error rate. Predicate execution and timer controls further distinguish a concrete defect from an alternative repair excluded by a narrower rubric. By preserving the links among contracts, answers, scores, and counterexamples, this study makes the observed benefit and its unresolved correctness questions jointly inspectable. Executable repair success and transfer beyond this plugin framework remain open.

Artifact Availability and Research Transparency

The working repository is <https://github.com/oh-my-dsh/dsh-plugin-upgrade-skill>. The focal archive is under `benchmark/results/artifacts/2026-09-15-glm-5.3-flash-unified-s16`; the supplementary review is under `paper/audit/output-review-20260917`. Original outputs and scores are preserved separately from review verdicts. Complete contract accounting and human-follow-up provenance are stored under `paper/audit/contract-review-20260917`. Offline mechanism probes and their execution environment are archived under `paper/audit/mechanism-checks-20260917`. The claim-to-evidence ledger identifies inputs and offline commands for regenerating the analyses. Availability is described per configuration; no universal raw-output completeness claim is made. A permanent release identifier and redistribution review remain author submission checks.

AI Assistance and Author Declarations

AI tools (Codex and ChatGPT) assisted with source inspection, analysis code, initial targeted output review, and manuscript drafting. The authors subsequently reported non-blind human checking by contributing plugin authors. The archive preserves both stages and does not claim independently measured human agreement.

Author identities, contributions, approval, funding, competing interests, and the final publisher-specific AI declaration must be confirmed by the authors before submission; this review draft does not assert those confirmations.

References

- Keyuan Cheng, Xudong Shen, Yihao Yang, Tengyue Wang, Yang Cao, Muhammad Asif Ali, Hanbin Wang, Lijie Hu, and Di Wang. CODEMENV: Benchmarking Large Language Models on Code Migration, 2025. URL <https://arxiv.org/abs/2506.00894v1>.
- Shahriar Golchin and Mihai Surdeanu. Time Travel in LLMs: Tracing Data Contamination in Large Language Models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.08493>.

- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?, 2026. URL <https://arxiv.org/abs/2603.15401v1>.
- Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills, 2026. URL <https://arxiv.org/abs/2605.23899v1>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-chung Lee. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks, 2026. URL <https://arxiv.org/abs/2602.12670v1>.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.03688>.
- Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL <https://arxiv.org/abs/2404.13076>.
- Tongtong Wu, Weigang Wu, Xingyu Wang, Kang Xu, Suyu Ma, Bo Jiang, Ping Yang, Zhenchang Xing, Yuan-Fang Li, and Gholamreza Haffari. VersiCode: Towards Version-controllable Code Generation, 2024. URL <https://arxiv.org/abs/2406.07411v2>.
- Ziyue Yang and Fan Ding. Signal or Noise? A Benchmark Study of Agent Skills in Web Development, 2026. URL <https://arxiv.org/abs/2608.23067v1>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2306.05685>.

A Task-level focal results

Table 3: Focal task-level original scores. Each arm has two repeats; differences use arm means. These are static rubric scores, not repair pass rates.

Task	No skill r1	No skill r2	Skill r1	Skill r2	Mean Δ
S1	55.00	60.00	100.00	100.00	42.50
S2	100.00	80.00	100.00	100.00	10.00
S5	87.50	100.00	100.00	100.00	6.25
S6	87.50	87.50	100.00	100.00	12.50
S9	100.00	100.00	100.00	100.00	0.00
S11	100.00	90.00	90.00	90.00	-5.00
S12	100.00	100.00	100.00	100.00	0.00
S13	100.00	100.00	100.00	100.00	0.00
S14	100.00	100.00	100.00	100.00	0.00
S15	100.00	100.00	100.00	100.00	0.00
S17	80.00	80.00	90.00	100.00	15.00
S18	100.00	100.00	100.00	90.00	-5.00
S19	100.00	100.00	100.00	100.00	0.00
S20	97.50	97.50	100.00	100.00	2.50
S21	100.00	100.00	100.00	100.00	0.00
S22	100.00	100.00	100.00	100.00	0.00

B Historical and supplementary context

B.1 Historical paired summaries

For each historical configuration, we use its recorded task reward on a 0–100 scale and preserve its original within-task aggregation. Qwen uses means of three scored attempts, DeepSeek and the two GLM groups use medians of three runs or rounds, and Terra uses one attempt. These numeric scales do not imply equivalent constructs: keyword grading, semantic judgment, and composite executable rewards differ. We do not pool configurations or regress their gains on their baseline scores as an independent capability measure.

Historical percentile intervals use 10,000 task-paired bootstrap samples with mulberry32 seed 20260907. The reported two-sided Wilcoxon signed-rank calculations exclude zero differences and use a tie-corrected normal approximation with continuity correction. These exploratory summaries can disagree near a boundary; neither is a substitute for validating the endpoint. No multiple-comparison-adjusted confirmatory family is claimed.

Historical mean changes range from -3.07 to $+10.67$ points (Table 4). The older Qwen configuration has substantial skill-arm timeouts under its local infrastructure. Terra excludes H8 because both arms have verifier timeouts. The GLM groups mix semantic and keyword judgments. These differences prohibit attributing cross-row variation to model capability alone. A contaminated Luna baseline, which accessed a native plugin-creator skill on H2 and H3, is excluded from this five-configuration comparison.

Within the historical GLM pair, the Flash-minus-5.2 difference in mean task lift is 6.23 points, with a paired bootstrap interval [2.59, 10.05]. The round-specific Flash gains are 6.32, 8.82, and 9.82, versus 1.55, 1.64, and 5.00 for GLM-5.2. Leaving one task out yields lift gaps from 5.33 to 7.00. These completed ro-

Model (n tasks)	Tasks	no-skill $\rightarrow$ with-skill	Mean paired Δ	95% CI	Signed- rank p	Protocol
qwen3.8-27b ($n = 56$)	full 56-task pool	44.67 $\rightarrow$ 41.60	-3.07	[-8.13, +1.99]	0.1771	3 scored, mean
deepseek-v4-flash ($n = 23$)	23-task snapshot	69.96 $\rightarrow$ 80.63	+10.67	[+0.48, +22.11]	0.1007	3-run median
gpt-5.6-terra ($n = 21$)	22-task pool, 21 scored	71.10 $\rightarrow$ 79.76	+8.67	[-4.29, +21.62]	0.2359	single-shot
glm-5.3-flash ($n = 22$)	S1-S22 static	77.77 $\rightarrow$ 87.05	+9.27	[+3.95, +16.09]	0.0056	3-round median
glm-5.2 ($n = 22$)	S1-S22 static	93.32 $\rightarrow$ 96.36	+3.05	[0.00, +7.18]	0.1378	3-round median

Judge identity: in both GLM groups the solver and the grader belong to the same model family — glm-5.3-flash graded its own runs, and 19 of the 22 glm-5.2 tasks were judged by glm-5.3-flash subagents (the remaining three by official keyword judges) — so correlated grader bias cannot be excluded (Section 2). glm-5.2’s CI lower bound touching zero alongside $p = 0.1378$ is a percentile-bootstrap boundary artifact: with 16 of 22 task deltas tied at zero, the lower endpoint of the resampling distribution sits at zero, so the interval brushing zero does not contradict the nonsignificant test.

Table 4: Task-level paired effect of the plugin-upgrade skill across five historical configurations, ordered by each group’s no-skill baseline (an outcome measure, not an independent capability metric; Section B.1). Each cell compares the two conditions on a 0–100 scale: per-task medians of three rounds (glm groups) or three runs (deepseek-v4-flash), per-task means of three scored attempts (qwen3.8-27b, reward means rescaled by 100), or single-shot rewards (gpt-5.6-terra; H8 excluded after verifier timeouts on both arms). Mean paired Δ is the mean of per-task with-skill-minus-no-skill deltas; 95% CIs are percentile intervals from 10,000 task-level paired bootstrap replicates (mulberry32, seed 20260907); p is the two-sided Wilcoxon signed-rank test (zero deltas excluded, tie-corrected normal approximation with continuity correction). Task pools and protocols differ across rows, so cross-row comparisons are descriptive.

business calculations support a descriptive contrast within the recorded protocols. Different judge mixes and development histories remain confounders; the contrast does not identify a capability-dependent response curve.

B.2 Newer supplementary configurations

A supplementary newer Qwen S16 run reports 71.17 to 77.89, a gain of 6.72 points, with a reported interval [-12.42, 26.25]. It contains 64 scored attempts, including five timeouts and two other nonzero-exit attempts. Termination status is distinct from the assigned score. Its raw answers and grading reasons are not archived, and the reported bootstrap randomization is insufficiently specified for exact interval reproduction. Fourteen tasks overlap with the focal Flash run; different hosts, budgets, judges, and selections prevent a controlled cross-model comparison. We use this result only as supplementary context.

Three supplementary GLM-5.3 rounds on S1–S22 are now archived. Recomputing all 132 verdicts against the pinned rubric, retaining half-point values before summation, gives round gains of 1.93, 3.18, and 6.59 points. The per-task median across rounds is 97.05 without the skill versus 98.64 with it, a gain of 1.59 points; its descriptive task-bootstrap interval is [-1.36, 5.80] (10,000 samples, seed 20260907). The third round used GLM-5.3 as judge rather than the GLM-5.3-Flash judge used in rounds one and two. The median cannot remove this protocol change, so these results remain a mixed-judge descriptive supplement.

Round three’s S17 no-skill answer receives zero because its affirmative immediate-registration recommendation triggers the rubric’s contradiction cap. The criterion decisions are retained: this pair contributes 100 of the round’s 145 total gain points. Excluding S17 descriptively leaves a 2.14-point mean gain across

21 tasks; this is a sensitivity, not a basis for removing the observation. The archive discloses quota-related relaunches, concurrency changes, and a truncated round-one answer. These rounds do not establish a capability ladder or restore comparability with older protocols.

B.3 Historical resource accounting

We also recomputed the earlier Flash round-one usage totals from its 22 sessions per condition. Fresh input totals are 599,882 and 678,101; output totals are 116,506 and 132,726; cache totals are 5,890,176 and 5,017,984. The recorded total-token sums are 6,606,564 and 5,828,811, respectively. These fields do not support an undifferentiated claim of a 2.2-fold token increase. Their accounting differs from the focal host’s token field, so resource ratios are reported within configurations only.

C Artifact map and reproduction scope

Historical result files are stored under `benchmark/results:paired-effect-stats.json` generates the paired table, and `glm-pair-stability.json` records GLM stability. The focal directory contains the selection, schedule, execution-order records, original reports and judgments, aggregate scores, paired analysis, and the earlier targeted review. The later review stores frozen selection and complete criterion verdicts separately. The script `paper/scripts/summarize-submission-evidence.mjs` checks report hashes, recomputes review scores, deduplicates overlapping answers, and regenerates sensitivity and historical resource summaries without model calls. Its `-check` mode verifies the committed generated summary.

A frozen 23-task historical inventory and its source metadata are retained in `paper/generated`; they are not substituted for the 16-task selection. Exposure information remains in `paper/audit/task-exposure-ledger.csv`. Missing raw artifacts, unverified model identities, and pending author declarations are recorded as limitations rather than filled in by inference.